

RESEARCH ARTICLE

A DSL-Driven Solution and Code Generation for Accelerated Tourism Decision Support System Development

ADNANE SOUHA^{ID}, (Member, IEEE), **CHARAF OUADDI**^{ID}, (Member, IEEE),
LAMYA BENADDI^{ID}, (Member, IEEE), **HAMZA ABDELMALEK**^{ID}, **EL MAHI BOUZIANE**,
AND ABDESLAM JAKIMI^{ID}, (Member, IEEE)

Software Engineering and Information Systems Engineering (GL-ISI) Team, Faculty of Sciences and Techniques of Errachidia (FSTE), University of Moulay
Ismail (UMI), Meknes 50050, Morocco

Corresponding author: Adnane Souha (ad.souha@edu.umi.ac.ma)

This work was supported by the Ministry of Higher Education, Scientific Research and Innovation, the Digital Development Agency (DDA), and the National Center for Scientific and Technical Research (CNIRST) of Morocco under Grant Alkhawarizmi/2020/32.

ABSTRACT Decision Support Systems (DSS) play a crucial role in commerce and tourism by assisting decision-making through data analysis, modeling, and visualization. However, their development is often complex, costly, and requires expertise in both software engineering and Artificial Intelligence, making them inaccessible to many organizations. To address these challenges, this paper introduces a Domain-Specific Language (DSL), a graphical decision rule modeling tool, and code generation templates designed to streamline DSS development for smart tourism. The proposed framework enables rule-based system modeling and automatic code generation, leveraging Drools, a powerful rule engine, to manage business rules, execute inferences, and optimize decision processes. This approach significantly reduces development time while improving system flexibility and adaptability to meet the diverse needs of smart tourism. A comparative study demonstrates a substantial reduction in development time, while a case study confirms the framework's ability to generate functional applications suited to real-world environments. Our contribution lies in automating and simplifying DSS development by integrating a dedicated DSL and Drools, reducing manual programming complexity, and ensuring the generation of optimized, maintainable, and easily integrable code. This enhances productivity, reduces development effort, and ensures adaptability to changing business needs.

INDEX TERMS Decision support systems, domain-specific language, smart tourism, rule-based systems, code generation.

I. INTRODUCTION

Decision Support Systems are integral to various domains, including commerce, tourism, healthcare, and environmental management, where they serve as critical tools for analyzing complex data and enhancing decision-making processes. In commerce, DSS facilitates the prediction of consumer trends, the optimization of inventory systems and the personalization of customer interactions, which helps achieve competitive advantages. In the tourism sector, they play

a pivotal role in forecasting traveler behavior, managing accommodation capacities, and refining marketing strategies to improve destination appeal and operational efficiency.

A DSS is an information system designed to organize and structure extensive data, assisting decision-makers in navigating complex decision-making scenarios [1]. The primary feature of DSS lies in their capacity to process and structure large volumes of data, enabling timely and well-informed decisions in dynamic and often uncertain environments. Recent advancements, particularly the integration of Artificial Intelligence (AI), Machine Learning (ML), and big data analytics, have greatly expanded the functionality of

The associate editor coordinating the review of this manuscript and approving it for publication was Asad Waqar Malik^{ID}.

DSS, making them more adaptive, precise, and intelligent in addressing complex challenges [2].

The development of DSS is a complex process that requires the integration of diverse techniques and frameworks to ensure their functionality and applicability. Several approaches to DSS development have been proposed, including AI-based methods, rule-based systems, and hybrid approaches, each aimed at enhancing the system's ability to address complex decision-making challenges.

Major technology companies provide various tools to support DSS development or deliver analysis and reporting solutions. Advanced DSS leverage AI for predictive and adaptive decision-making, as seen in platforms like Azure Machine Learning [3] and IBM Decision Optimization [4]. Alternatively, rule-based systems such as Drools [5] automate decision processes using predefined logic. Meanwhile, analysis and reporting tools like Tableau [6] and Power BI [7] are designed for data analysis and visualization to support operational decisions, though they lack the capabilities required for addressing complex strategic decision-making scenarios.

Despite these notable advancements, the development of DSS remains a resource-intensive and time-consuming process. The complexity of constructing sophisticated systems that integrate diverse data sources, decision models, and advanced technologies demands significant investment in resources and specialized expertise. Consequently, organizations aiming to implement DSS face considerable challenges in managing both development costs and timelines effectively.

To address these challenges, there is a need for innovative tools and methodologies that streamline and simplify the DSS development process. In the context of smart tourism, decision-making often involves localized rules that reflect the unique characteristics of a destination, such as seasonal demand, cultural events, or visitor profiles. These rules are typically managed by tourism professionals who may not have technical backgrounds. To address this, we propose a DSL tailored to the tourism sector, with the goal of empowering non-technical stakeholders, such as destination managers, to model and update decision rules autonomously, without requiring direct involvement from software developers. This autonomy not only enhances flexibility and responsiveness but also reduces the long-term maintenance cost of decision support systems.

This article presents a framework designed to facilitate the modeling and implementation of DSS, specifically within the tourism sector. The framework integrates a DSL for decision rule modeling, a graphical tool, and an automated code generator. By leveraging Drools, a powerful rule engine, it enables the formalization and execution of decision-making logic and automatically generate optimized and maintainable code. This approach significantly reduces development time and costs while ensuring flexibility, maintainability, and seamless integration to meet the evolving decision-making needs of the tourism industry. The effectiveness of this

approach is validated through a comparative study with traditional development methods, demonstrating substantial improvements in development efficiency and code quality, as well as through a case study of its applicability in real-world scenarios.

This paper is organized as follows. Section II provides the foundational background, introducing essential concepts related to smart tourism, DSS, and DSLs. Section III reviews the existing literature in these domains, identifying key gaps that the proposed approach seeks to address. In Section IV, we detail the methodology, including the development of the DSL, the graphical modeling tool, and the code generator. Section V illustrates the practical application of the approach through a case study in the tourism sector. Section VI evaluates the effectiveness of the proposed method by comparing it with traditional DSS development techniques. Section VII discusses the current limitations of the approach and highlights key challenges for future development. Finally, Section VIII concludes the paper by summarizing the findings and outlining directions for future research.

II. BACKGROUND

The development of intelligent systems for smart tourism requires software approaches that can address the domain's complexity, variability, and the involvement of non-technical stakeholders. In this context, Model-Driven Engineering (MDE) offers high-level abstraction mechanisms to design flexible and maintainable systems, while DSLs empower domain experts to model and manage decision logic without programming expertise. When applied to DSS, these techniques help formalize rules and automate code generation, enabling responsive and adaptable solutions for dynamic tourism scenarios. This section provides an overview of the foundational concepts relevant to the proposed approach, including smart tourism, DSS, MDE, Model-Driven Architecture (MDA) and DSLs.

A. SMART TOURISM

Smart Tourism integrates cutting-edge technologies, including the Internet of Things (IoT), AI, Big Data, and mobile applications, to improve travel experience and enhance destination management. This approach aims to deliver tailored services to tourists while emphasizing sustainability and inclusivity. The core tenets of smart tourism encompass connectivity, enabling real-time access to information for travelers; accessibility, ensuring equitable service delivery for all, including individuals with disabilities; and sustainability, striving to mitigate the environmental impact of tourism activities.

To realize these principles, a variety of technologies play a crucial role. Mobile applications, for instance, deliver personalized recommendations and real-time contextual information to users, enhancing the overall travel experience [8]. Likewise, the IoT enables the proactive monitoring and management of tourism infrastructure, facilitating the optimization of visitor flows and resource utilization [9].

Furthermore, advanced tools such as chatbots and DSS are central to smart tourism. AI-powered chatbots provide real-time assistance and personalized interactions, thereby improving tourist engagement [10]. DSS, on the other hand, makes use of Big Data and AI to analyze complex datasets, optimize resource allocation, and enhance operational efficiency [11].

In that regard, Smart tourism offers numerous benefits, including improved tourist experiences, better resource optimization, and greater sustainability through intelligent management of infrastructure and services. However, this model also faces significant challenges, such as issues related to data privacy, cybersecurity, and inclusivity, particularly in regions with limited access to advanced technologies [12].

B. DECISION SUPPORT SYSTEMS

1) OVERVIEW OF DSS

DSS are interactive, computer-based tools designed to assist decision-makers in resolving complex or unstructured problems [13]. These systems provide access to data, models, and knowledge, enabling efficient analysis, enhancing decision-making processes [14], and supporting the identification of optimal solutions. By integrating resources such as databases, documents, and intuitive interfaces, DSS has become indispensable in various applications.

One of the most critical features of DSS is their capacity to organize and structure large volumes of information. This ability empowers decision-makers to make informed choices in intricate scenarios. Furthermore, DSS relies on business rules and decision logic to automate, rationalize, or guide decisions. Business rules, defined as predefined policies or guidelines, specify required actions under given conditions. These rules may be static or dynamic, which allows DSS to adapt effectively to changing parameters and evolving data inputs.

In addition to these foundational features, recent advancements have significantly expanded the capabilities of DSS. Intelligent systems (IDSS) have emerged, leveraging AI, ML, Big Data analytics, and ontologies to simulate and enhance human decision-making. As a result, modern DSS are more adaptive, precise, and efficient, enabling them to address dynamic and uncertain environments with greater accuracy.

Given these advancements, the applications of DSS have become increasingly diverse. They are now utilized across a wide range of sectors, including industry [15], healthcare [16], investment management [17], agriculture [18], disaster management [19], transportation services [20], and tourism [14], [21], [22], [23]. The growing adoption of DSS highlights their versatility and underscores their critical role in addressing complex decision-making challenges across various domains.

2) DSS CLASSIFICATION

In the literature, DSS are classified based on various criteria, each emphasizing specific aspects of their design

or application. Common classification criteria include the system's relationship to the user, the mode of assistance it provides, the scope of its use, its orientation, its focus, its capabilities, and the frequency of decisions it supports. For the purposes of this study, we adopt a classification based on orientation. According to this approach, DSS are categorized into six distinct classes as follows [24]:

- **Text-oriented DSS:** These systems support the decision-making process by organizing and storing text-based data that can be accessed, evaluated, and utilized by decision-makers.
- **Database-oriented DSS:** These systems centralize data from multiple sources, providing decision-makers with tools to manipulate and analyze large datasets effectively, thereby enhancing decision support.
- **Rule-oriented DSS:** Leveraging elements of AI, these systems utilize predefined rules and knowledge bases to generate solutions, making them effective for structured decision-making tasks.
- **Solver-oriented DSS:** These are specialized systems that employ solvers to analyze and address decision-making challenges in specific domains, such as financial analysis or optimization problems.
- **Spreadsheet-oriented DSS:** These systems enable decision-makers to develop and analyze models, providing capabilities for data manipulation and modeling in a familiar spreadsheet environment.
- **Compound-oriented DSS:** By integrating multiple approaches from the aforementioned categories, these systems offer enhanced flexibility and adaptability for analyzing and solving complex decision problems.

3) DSS ARCHITECTURE

DSS are built on a modular architecture consisting of four main components, as illustrated in Figure 1. These components work together to provide a robust framework for supporting decision-making processes.

- **Model Base (MB):** The Model Base is responsible for organizing and managing decision-making models that are used to analyze, simulate, and predict scenarios based on available data. For example, in finance, these models can project cash flows or evaluate the impacts of strategic decisions [25]. The MB offers users significant flexibility by enabling the selection, modification, or customization of models to meet specific analytical needs. Furthermore, it incorporates advanced algorithms, including decision trees, multi-criteria analysis, ML algorithms, probability density functions, maximum likelihood estimation, neural networks, and k-means clustering, all aimed at optimizing decision-making efficiency [26].
- **Knowledge Base (KB):** The Knowledge Base is an essential component that centralizes structured knowledge, rules, and heuristics to guide decision-making. In the context of rule-based DSS, this is where the

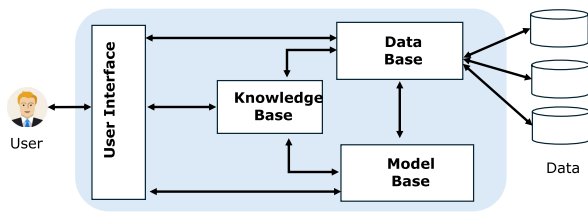


FIGURE 1. DSS architecture (adapted from [27]).

decision rules are defined, managed, and executed. It integrates contextual information and business logic and can utilize the outcomes of analytical models or ML algorithms to refine and adapt decision rules dynamically.

- **Database (DB):** The Database serves as the repository for both raw and processed data required for analysis. It includes various data types relevant to decision-making, such as booking information, visitor reviews, and reports on the utilization of tourism services.
- **User Interface (UI):** The User Interface represents the interaction point between the system and its users. It provides visualization tools, such as graphs, tables, and interactive simulations, which make complex data more accessible and easier to interpret for decision-makers.

4) DSS TECHNIQUES

DSS employs a range of advanced techniques to analyze complex data and deliver relevant recommendations. These techniques are primarily associated with the Model Base and Knowledge Base components of the DSS architecture (Figure 1). The Model Base integrates analytical and computational models, such as those based on AI or multi-criteria methods, while the Knowledge Base incorporates rule-based logic and domain expertise to guide decisions. These approaches can be categorized based on their underlying methodologies and applications, as outlined below:

- **AI-based Techniques:** AI-driven methods include ML algorithms such as neural networks, decision trees, and clustering techniques. These can identify patterns in data and provide predictive or prescriptive insights. Advanced DL models and reinforcement learning are also employed to address complex decision-making scenarios and offer enhanced accuracy and adaptability [28].
- **Multi-Criteria Decision Making (MCDM):** MCDM approaches support the evaluation and ranking of alternatives based on multiple, often conflicting, criteria. For example, the Analytic Hierarchy Process (AHP) structures decision problems into hierarchical levels to prioritize alternatives [29]. Similarly, the Technique for Order Preference by Similarity to Ideal Solution (TOPSIS) ranks options by their closeness to an ideal solution [30], while the ELECTRE method

focuses on analyzing outranking relationships between options [31].

- **User-Centered Approaches:** Modern DSS emphasizes user-centered designs to enhance usability and ensure seamless integration into workflows. Explainable AI (XAI) is a key feature that builds user trust by making system recommendations transparent and understandable [32]. Additionally, interactive dashboards provide real-time, tailored insights, while systems that offer personalized recommendations adapt to users' preferences and behavior, further enhancing the decision-making experience [33].
- **Rule-Based Approaches:** Rule-based systems rely on engines and domain-specific knowledge to simulate expert decision-making. These systems use logical inference mechanisms and predefined domain rules to generate decisions or recommendations [34]. Ontologies and knowledge graphs further enhance these systems by structuring and interlinking data, enabling context-aware decision-making [35].
- **Hybrid Approaches:** Hybrid DSS combines multiple decision-making techniques to improve accuracy, adaptability, and overall effectiveness. By integrating the strengths of different methods, hybrid systems address the limitations of individual approaches. For instance, combining the Analytic Hierarchy Process (AHP) with the Simple Additive Weighting (SAW) method has gained prominence for its robust decision-making capabilities [36].

5) DSS DEVELOPMENT FRAMEWORKS

A diverse range of tools and services is available from leading technology companies to support the development of DSS and Business Intelligence (BI) solutions. These tools can be categorized based on their primary function and analytical focus [31, 32]. Each category addresses different aspects of organizational decision-making needs.

- **Tools for Advanced Analytical and Predictive DSS:** These tools focus primarily on complex analytical processes, predictive modeling, and optimization capabilities and can be divided into two main categories:
 - **AI-Based Tools:** These tools harness ML and AI to facilitate predictive and adaptive decision-making. Prominent examples include:
 - * Microsoft Azure Machine Learning [3]: Supports intelligent automation for complex workflows.
 - * IBM Watson Decision Platform [4]: Provides solutions for advanced decision-making processes.
 - * Google Cloud AI and AutoML: Delivers accessible and scalable AI-driven analytics.
 - * Amazon SageMaker: Enables advanced analytical capabilities for large-scale decision models.

* **Meta PyTorch and Horizon:** Meta's PyTorch and Horizon frameworks focus on real-time adaptive modeling using deep learning approaches [37]. These frameworks are particularly effective at deriving actionable insights from complex data streams, supporting advanced decision-making in dynamic environments [38].

– **Rule-Based Tools:** These systems use predefined business rules to automate decision-making through logical inference [39]. A leading example is Drools, an open-source business rules management system. Drools efficiently manage complex decision-making logic by employing rule-based reasoning, making it a preferred choice for organizations requiring structured, repeatable decision processes [40].

• **Tools for Data Visualization and Reporting-Oriented DSS** These tools specialize in data organization, analysis, and visualization, empowering users to derive actionable insights from complex datasets [36]. Their primary strength lies in data exploration, interactive reporting, and visualization capabilities [37]. Prominent examples include:

- Tableau [6]: Recognized for its interactive visualizations and comprehensive reporting capabilities.
- Microsoft Power BI [7]: Provides dynamic dashboards and robust tools for exploratory data analysis [38].
- QlikView [41]: Offers intuitive interfaces for in-depth data analysis and visualization.

In contrast to general-purpose DSS platforms such as IBM Watson, Google AutoML or Azure Machine Learning, which focus primarily on AI-based decision-making and require data science expertise, our approach introduces a lightweight, rules-based alternative suitable for domain experts. Although these tools offer robust predictive capabilities, they often operate as closed-box systems with limited transparency and customization of the decision logic. In our framework, decision rules are explicitly modeled and maintained via a DSL, guaranteeing clarity, traceability and adaptability. In addition, our solution reduces development complexity by enabling non-technical users to define rules and generate code automatically without relying on learning AI models or integration overheads.

C. DROOLS AND RULE-BASED SYSTEMS

Drools is a Business Rule Management System (BRMS) that allows businesses to define, manage, and execute business rules independently from application code. It is an open-source rule engine written in Java and is used for decision automation, complex event processing, and business logic implementation. Drools is based on the Rete algorithm, which optimizes rule matching and execution for high performance. It enables dynamic decision-making by

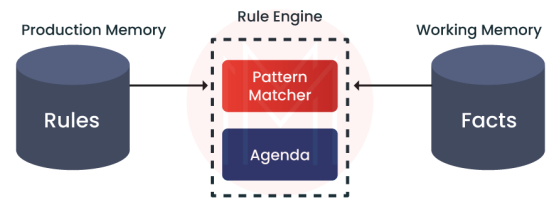


FIGURE 2. Architecture of the drools rule engine (from [42]).

separating business rules from application logic, making the system more maintainable and scalable. Figure 2 illustrates the architecture of the Drools rule engine, which is based on a modular approach designed to optimize the execution of business rules and ensure automated and efficient decision-making. This architecture consists of three fundamental components: Production Memory, the Rule Engine, and Working Memory, which dynamically interact to guarantee a rigorous application of the defined rules.

- Production Memory serves as a storage space dedicated to business rules. These rules are formalized in various formats, including Drools Rule Language (DRL) files, decision tables, and guided rules. By decoupling business rules from application code, this mechanism enhances the maintainability and scalability of information systems, allowing rules to be modified without requiring application redeployment.
- Working Memory, on the other hand, acts as a temporary repository where facts, business data on which the rules from Production Memory are applied, are stored. These facts are dynamic and can be updated based on interactions with the rule engine. This flexibility enables real-time adaptation of automated decisions according to evolving data.
- The Rule Engine, the central element of this architecture, is composed of two main subcomponents: the Pattern Matcher and the Agenda. The Pattern Matcher implements the Rete algorithm, an optimization model designed to efficiently compare facts with rule conditions while reducing redundant calculations. When a fact satisfies a rule's conditions, the rule is activated and transferred to the Agenda, which manages the order of rule execution according to various prioritization strategies, such as salience, rule activation order, or hierarchical group organization.

The functioning of this architecture is based on an iterative process. The insertion of facts into Working Memory triggers an automatic evaluation by the Rule Engine, which matches these facts against the conditions defined in Production Memory. The applicable rules are then activated and placed in the Agenda, awaiting execution. Once executed, these rules may modify existing facts or introduce new ones, leading to a re-evaluation of conditions and potentially activating other rules. This cycle continues until no further rules can

be applied, marking the completion of the rule engine's execution.

D. MDE AND MDA

Model-Driven Engineering and Model-Driven Architecture are foundational paradigms aimed at modernizing and enhancing the software development process. These approaches reduce code dependency by emphasizing the use of abstract models, enabling greater automation and adaptability in system development [43].

1) MODEL-DRIVEN ENGINEERING (MDE)

MDE focuses on abstraction and automation in software development and is structured around three core elements [44], [45]:

- **Models:** These describe systems at an abstract level, often represented through diagrams or visual constructs.
- **Metamodels:** These establish the rules and structures that models must adhere to, ensuring consistency and standardization [46].
- **Model Transformations:** These involve converting a model into another model or directly into source code, based on predefined rules and processes.

The primary objective of MDE is to reduce manual and repetitive tasks by automating key development steps through transformation tools and engines. This approach promotes the reuse of concepts, minimizes human error, and lowers associated costs [47]. MDE has demonstrated significant utility across diverse fields, including healthcare [48], smart tourism [49], and critical systems such as those in the nuclear [50] and aerospace sectors [51]. Despite its advantages, the adoption of MDE faces challenges, including difficulties in integrating with existing systems, limited availability of tools, and the need for development teams to adopt and adapt to this approach effectively [52].

2) MODEL-DRIVEN ARCHITECTURE (MDA)

On the other hand, MDA, proposed by the Object Management Group (OMG), is a specific implementation of MDE that emphasizes standardization and the automatic generation of code from models [53]. MDA introduces three foundational concepts:

- **Computation-Independent Models (CIMs):** These models focus on the environment and requirements of the system, without considering the system's computational aspects.
- **Platform-Independent Models (PIMs):** These models describe system functionalities at a high level of abstraction, without reference to the technical specifics of any particular platform.
- **Platform-Specific Models (PSMs):** These models are derived from PIMs by incorporating the technical details required for implementation on a specific platform.

By promoting the separation of concerns, MDA enables developers to focus on the system's abstract design while

automating much of the code generation process. This approach significantly enhances the efficiency of software development across various technological environments.

MDA has been successfully applied in numerous domains, such as automating software test generation, which ensures the creation of reliable and consistent test cases [54].

3) MODEL TRANSFORMATIONS

By capitalizing on the automated code generation capabilities of MDA [55], model transformations serve as a key mechanism for translating abstract PIMs into platform-specific implementations. Such transformations ensure that the generated code complies with the target platform's requirements while preserving alignment with the original high-level design.

In this context, different techniques for model transformation are adopted, each distinguished by how the transformation rules are defined [56]. First, a programming-based method uses object-oriented programming languages to manually implement transformations. Second, a template-based method, exemplified by tools like Acceleo, employs parameterized templates, yielding reusable and efficient transformations. Finally, a modeling-based method applies model engineering directly to transformations, often through standards like MOF QVT (Meta-Object Facility Query/View/Transformation) [57] or languages such as ATL (Atlas Transformation Language) [58], thereby fostering independent and productive transformation models [59].

In our research, we employed Acceleo, a source code generator provided by the Eclipse Foundation and designed specifically to implement the MDA framework. Through its template-based paradigm, Acceleo enables the automated generation of executable code from PIMs by applying predefined templates.

E. DOMAIN-SPECIFIC LANGUAGE (DSL)

DSLs are developed to supply domain experts with specialized abstractions that target the intricate challenges characteristic of particular fields. They play an essential role in MDE by serving as a bridge between domain specialists and software engineers. Within an MDE framework, DSLs commonly define PIMs, which are subsequently transformed into PSMs or directly into executable code using model transformation processes.

A typical DSL architecture includes four key elements: an abstract syntax, representing the conceptual structure of the domain; a concrete syntax, outlining the textual or graphical notation for expressing those concepts; a semantics, which establishes the operational interpretation and behavior of the defined constructs, and a serializable syntax that ensures the DSL constructs can be saved and transferred in a consistent format. In the realm of chatbot development [60], for example, DSLs facilitate the representation of conversational flows and intent-response relationships, which can then be converted into deployable applications for various messaging

platforms [61], [62]. Similarly, in recommendation systems, DSLs can be used to formalize user preferences and filtering criteria, which streamlines the generation of personalized suggestions in various fields such as tourism [63].

III. RELATED WORK

DSS development frequently involves intricate processes that necessitate close collaboration between domain experts and software engineers. This complexity is further amplified by the diverse requirements and technologies prevalent in various sectors. In response to these challenges, MDE has gained traction as a robust approach, offering methodologies and tools to conceptualize, automate, and streamline DSS design.

Leveraging DSLs, MDE seeks to shorten development cycles, decrease reliance on advanced technical expertise, and bolster systems' adaptability to changing contexts. Recent research has examined these principles in a range of domains, including logistics [64], healthcare [65], and analytical visualization [66], thereby affirming the effectiveness of MDE-oriented techniques.

Khakpour et al. [66] introduce ModelViz, a DSL that harnesses MDE to construct analytical visualization systems. Their framework emphasizes a multi-layered architecture that unifies heterogeneous data, business imperatives, and visualization technologies. Through application in complex supply chains, the authors illustrate how MDE can substantially reduce development time while addressing distinct business requirements.

Similarly, Saxena et al. [65] present MediK, a DSL focused on executable clinical guidelines for improving clinical decision support systems (CDSS). By furnishing formal analysis capabilities and ensuring consistent guideline evolution, this language provides a compelling example of how MDE can resolve implementation challenges associated with the complexity and frequent updates of clinical protocols.

Ulitin and Babkin [64] investigate an ontology-based DSL for technical planning in dynamic operational settings. Their strategy stands out by facilitating real-time reconfiguration of the DSL, adapting to shifting domain conditions and varying user skill levels, thus enhancing DSS flexibility.

Mazón and Trujillo [67] propose an MDA-driven approach to Data Warehouse (DW) development, focusing on the transformation of PIM models into PSM using UML profiles and QVT-based transformations. This method aspires to provide a cohesive and standardized solution for designing critical DW components, including ETL procedures, data sources, and repositories.

Meanwhile, Erwig and Kumar [68] introduce MADMAX, a DSL embedded in Haskell, aimed at multi-attribute decision-making. Their approach prioritizes explainability, enabling users to produce detailed evaluations that elucidate why a particular choice outperforms other potential alternatives.

In contrast to these studies, our work addresses the tourism sector, which remains underrepresented in MDE-based

DSS research. By automating decision logic formalization and code generation, our framework significantly reduces development time and costs while ensuring flexibility, maintainability, and seamless integration of DSS into smart tourism platforms.

IV. DESCRIPTION OF THE APPROACH

This section presents the main contributions of our approach: (1) a domain-specific metamodel capturing decision-related and tourism concepts; (2) a DSL and graphical modeling tool allowing users to define DSS models visually; and (3) an automated code generator that produces executable applications integrated with Drools. The section first provides an overview of the MDE-driven process, then details each contribution and its implementation.

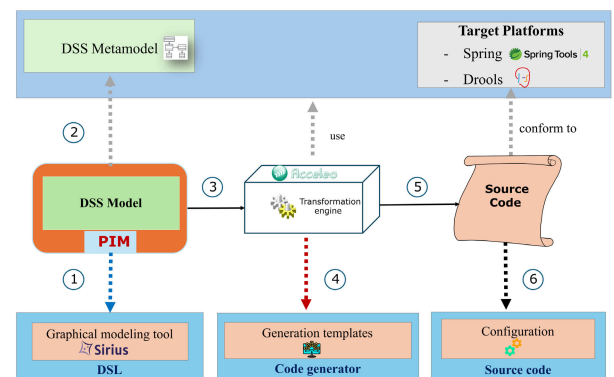


FIGURE 3. Overview of the approach.

A. OVERVIEW OF THE APPROACH

Figure 3 presents our MDE-driven methodology for advancing rule-based DSS in the tourism sector. This approach incorporates a tailored DSL and a corresponding meta-model, complemented by a graphical modeling environment that offers drag-and-drop functionality through icon-based interfaces. The framework produces Java-based backend code aligned with the Model-View-Controller (MVC) pattern and exposes RESTful APIs for interaction with client applications. Its architecture is rooted in Java EE standards, leverages the Spring framework for structural organization, and employs Drools for managing decision rules.

To improve readability and clarify component interactions, the figure is annotated with step numbers, which represent the key stages of the model-driven code generation workflow:

- 1) **Model creation:** The user creates the DSS model (PIM) using the graphical modeling tool, defining domain entities and decision rules.
- 2) **Model conformance:** The PIM is an instance of the DSS metamodel, ensuring that the model adheres to the domain-specific abstract structure.
- 3) **Model transformation input:** The PIM is passed as input to the transformation engine.

- 4) **Template-based code generation:** The transformation engine (Acceleo) applies predefined generation templates to produce platform-specific code.
- 5) **Code output:** Source code is generated, conforming to target platforms such as Spring and Drools.
- 6) **Code configuration:** The generated source code is finalized and adapted through platform-specific configuration.

B. CONCEPTUAL DESIGN AND MODELING

1) METAMODEL

The proposed metamodel provides a high-level structure for modeling decision-making processes in the context of smart tourism. It is an original contribution of this work, developed using a domain-driven design approach grounded in two complementary pillars:

- 1) A domain model for smart tourism, published by our team in [69], based on the 6As framework introduced by [70]. This model offered a comprehensive conceptualization of the tourism ecosystem, including attractions, accommodations, services, and activities. By analyzing this model, we identified the core domain entities and their interrelations, which were abstracted into reusable concepts (e.g., SmartTourismElement, Attraction, Amenity, Service, etc.).
- 2) A conceptual analysis of Drools, a widely used rule engine, which informed the design of our decision layer. Constructs such as Decision, Rule, Condition, Operator, and Action were derived from Drools semantics and integrated into the metamodel. This integration ensures compatibility with rule-based reasoning engines and supports flexible decision automation.

The resulting metamodel enables tourism destination managers to oversee various sector-specific elements while embedding the capability to define and evaluate business rules. Logical operators allow for fine-grained control of decision conditions, which trigger context-appropriate actions. Figure 4 illustrates this conceptual architecture. To develop and validate the metamodel, we used Ecore Tools and Sirius, both part of the Eclipse Modeling Framework (EMF). These tools facilitated the graphical modeling and ensured the formal correctness of structural relationships between elements. Although instantiated in the tourism domain, the metamodel was designed to be domain-agnostic at its core. Elements such as Decision, Rule, and Condition are abstract and reusable across various domains (e.g., healthcare, finance, logistics), requiring only the redefinition of domain-specific subclasses to adapt the DSL accordingly.

2) FUNCTIONAL MODEL OF DSS

To address the complexity and variability of modern DSS, it is essential to define functionalities in a modular and flexible way. This allows systems to adapt to the specific needs of users and organizations. Some functions are essential for core operation, while others may be optional or configurable

depending on business requirements. This modularity enables the combination of different capabilities and supports the customization of DSS for diverse application domains.

In response to this need, we developed a feature model that organizes DSS capabilities into configurable categories. The model, presented in Figure 5, was constructed based on a synthesis of established frameworks in DSS literature and principles from software product line engineering. It builds on prior research that identifies key subsystems, such as presentation, knowledge, and problem-processing components [71] and classifies functionalities according to the types of decision-making support they provide [72]. To capture variability and configuration options, the model adopts feature modeling techniques inspired by the Feature-Oriented Domain Analysis (FODA) method [73], distinguishing between mandatory, optional, and alternative features.

Figure 5 provides a functional perspective that complements the structural architecture illustrated in Figure 1. While Figure 1 presents the general system components typically found in DSS, Figure 5 focuses on the operational capabilities that can be enabled, configured, or extended in smart tourism applications. It highlights how functional variability supports flexibility, scalability, and domain-specific customization in DSS design.

The model is structured around five main categories, each detailing specific features aimed at ensuring an efficient and adaptable DSS.

- **Data Management:** This category encompasses essential functionalities for data collection, storage, and processing. While data collection and data storage are mandatory, data processing is an optional feature, improving the quality of information used for decision-making.
- **Model Management:** This category includes capabilities for executing and optimizing models. While model execution is mandatory, model tuning is optional. Additionally, users must make an alternative choice between predefined models and custom models, ensuring flexibility based on analytical needs.
- **User Interface:** This category ensures interaction between the DSS and its users. The dashboard is a mandatory feature, while report generation is optional. For data visualization, users can choose among multiple options (charts, tables, or maps) within a group relationship, allowing customization according to analytical preferences.
- **Decision Support:** This section includes advanced functionalities that enhance decision-making. Rule-based reasoning, ML models, and optimization are all optional features, allowing the DSS to adapt to various scenarios and levels of complexity.
- **Security & Access Control:** Security is crucial, making authentication and authorization mandatory to ensure restricted access for authorized users. Logging and auditing, on the other hand, are optional, providing an

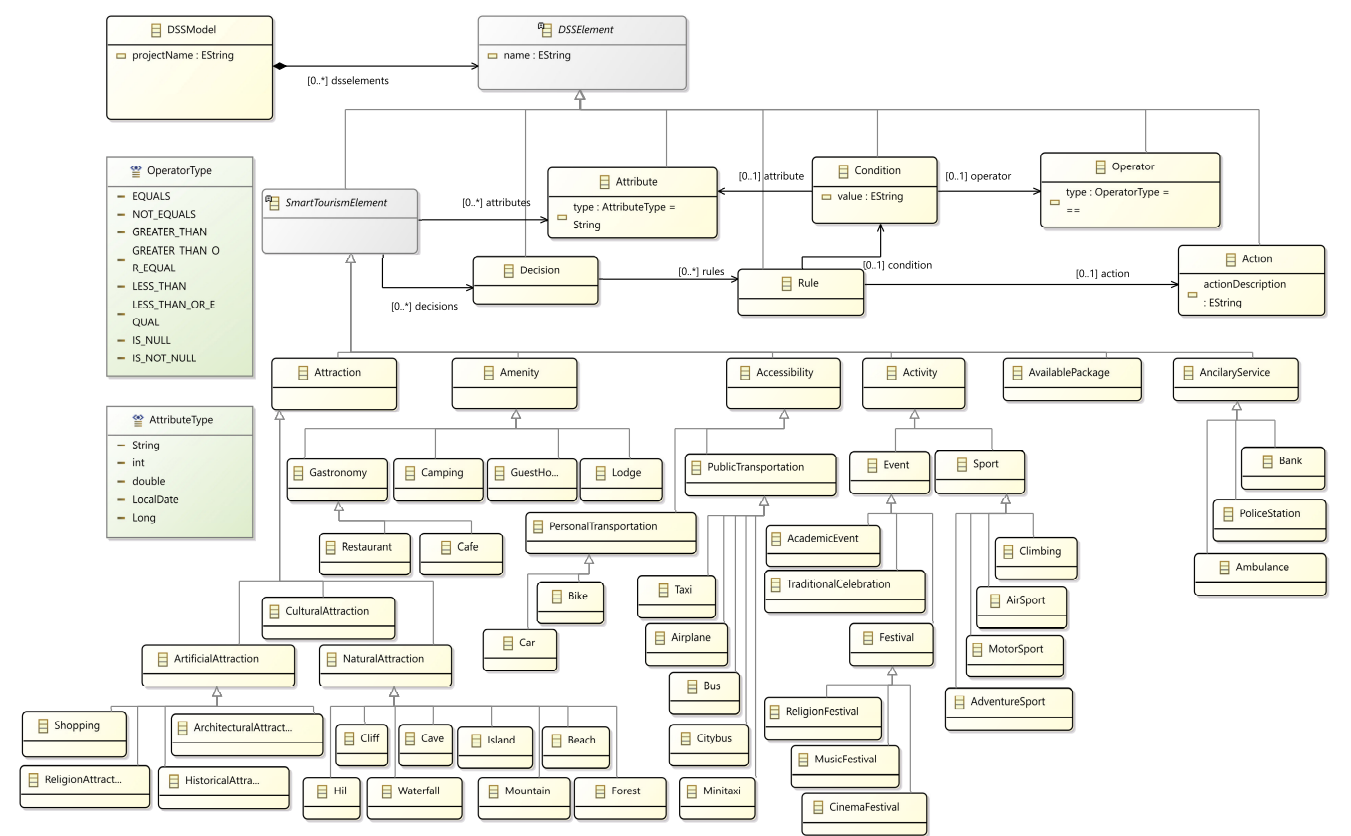


FIGURE 4. Conceptual representation of the metamodel for tourism DSSs.

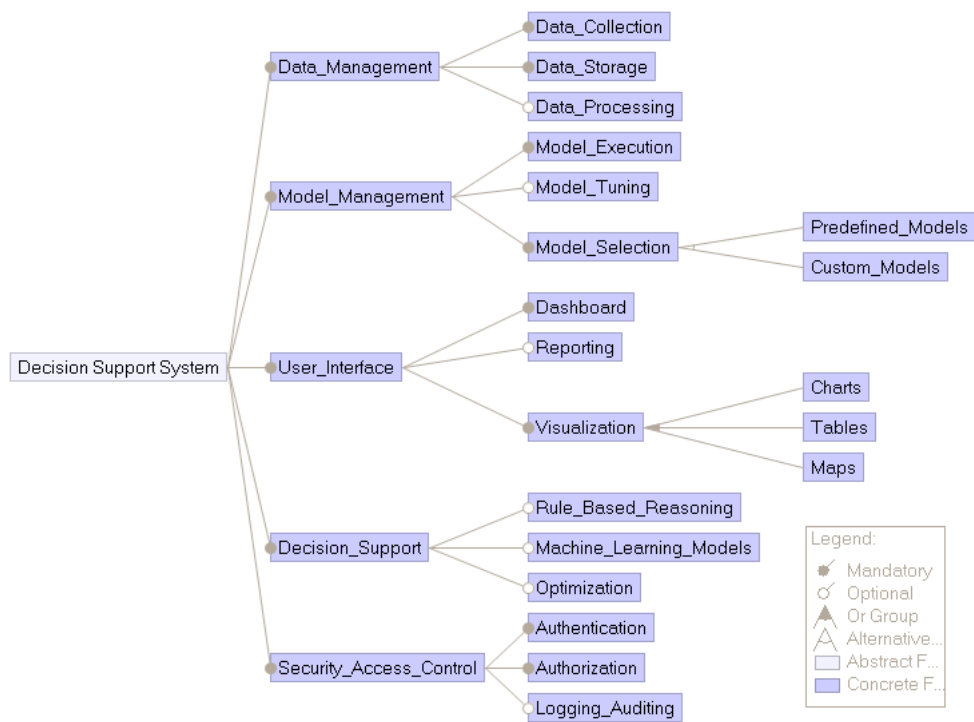


FIGURE 5. Feature model of a DSS representing its hierarchical structure and key functionalities.

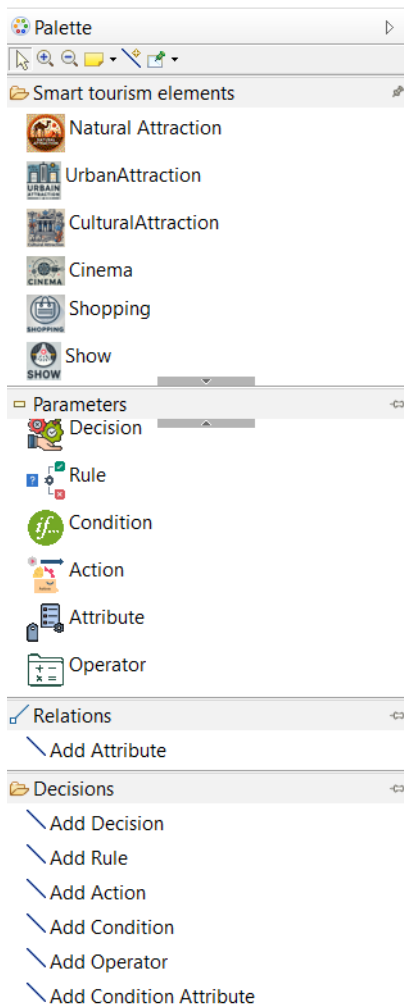


FIGURE 6. Modeling tool palette.

additional layer of monitoring and traceability of system activities.

3) GRAPHICAL MODELING TOOL

As part of this investigation, we developed a graphical modeling environment using Sirius to support visualization of DSS components and their interconnections within a diagrammatic interface. This tool also enables the direct creation of new elements via a dedicated tool palette. The palette is organized into several primary categories (see Figure 6):

- **Smart tourism elements:** Encompasses various tourist features, including natural, urban, and cultural attractions, cinemas, shopping areas, and similar sites.
- **Parameters:** Consolidates system-related components, such as decisions, rules, conditions, actions, attributes, and operators.
- **Relations:** Facilitates linkages between smart tourism elements and their associated attributes.

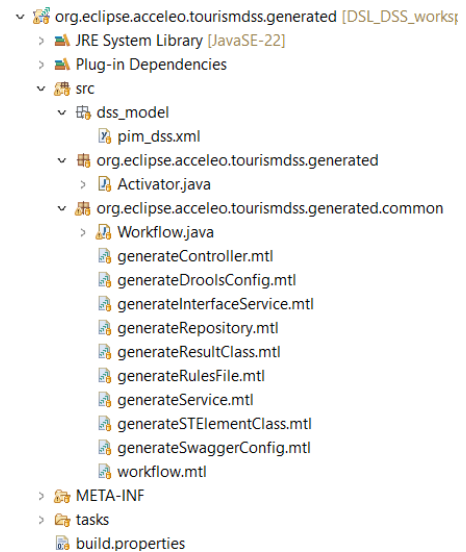


FIGURE 7. Templates for code generation.

- **Decisions:** Offers functionalities for adding and configuring decisions, rules, actions, conditions, operators, and conditional attributes.

An important advantage of our DSL-based approach lies in its flexibility: business rules are modeled independently from the implementation code, allowing users to modify them directly within the graphical modeling environment. This design enables quick updates and continuous evolution of the DSS logic without altering the underlying DSL structure. Business rules can be updated at any time by modifying their condition or action properties through the graphical editor. Such changes are instantly reflected in the model and automatically incorporated into the generated code during the next build cycle, without requiring DSL reconfiguration or manual coding effort.

C. CODE GENERATION AND TARGET PLATFORM

1) CODE GENERATOR

In addition, we incorporated a code generator built on a collection of predefined templates (see Figure 7), designed to automate the production of code from the specified DSS models. Code generation templates serve as predefined structures that streamline coding tasks by creating targeted code from high-level models, which is essential in MDE scenarios where models must be systematically converted into executable components. In our work, these code generation templates were crafted using Acceleio, a model transformation language. Acceleio implements the MOFM2T (Meta-Object Facility Model to Text) standard defined by the OMG, thereby aligning with recognized best practices in model-driven engineering. Figure 8 shows the template used to generate a rules file using MTL in Acceleio. This template automates the creation of rules by extracting information from a DSSModel and a SmartTourismElement. The template contains generation structures, which define the layout and

```

generateRulesFile.mtl x
1 [comment encoding = UTF-8 /]
2 [module generateRulesFile('http://www.example.org/tourismdss')]
3
4 [template public generateRulesFile(aDSSModel : DSSModel, smartTourismElement: SmartTourismElement)]
5
6 [file ('/rules/' .concat(smartTourismElement.name.toLowerFirst()).concat('Rules.drl'), false, 'UTF-8')]
7 package com.example.[aDSSModel.projectName/].rules;
8
9 import com.example.[aDSSModel.projectName/].model.[smartTourismElement.name/];
10 global com.example.[aDSSModel.projectName/].model.Result result;
11
12 [for (decision : Decision | smartTourismElement.decisions)]
13 [for (rule : Rule | decision.rules) ]
14 rule "[rule.action.actionDescription/]"
15 when
16   $[smartTourismElement.name.toLowerFirst()/]: [smartTourismElement.name/][rule.condition.attribute.name/]
17   [rule.condition.operator.type.toString()/] [rule.condition.value/]
18 then
19   result.addMessage("[rule.action.actionDescription.toUpper()/]");
20 end
21 [/for]
22 [/for]
23 [/file]
24
25 [/template]

```

FIGURE 8. Code snippet from a template to generate a drools file.

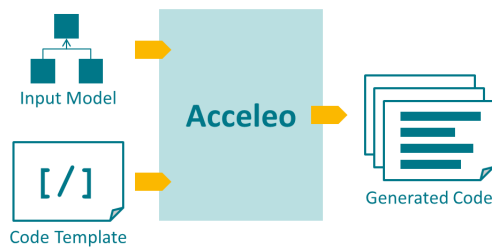


FIGURE 9. Aceleo-based code generation process.

structure of the generated file, and queries, which extract, filter and manipulate model data to customize rules. Figure 9 illustrates the overall code generation process using Aceleo. The process begins with an input model, which represents the system structure and behavior. This model is processed using a code template, defined in MTL, to transform high-level abstractions into concrete, executable code. The Aceleo engine applies the template logic to the model, producing automatically generated code that adheres to the specified rules and design patterns.

2) TARGET PLATFORM

The code generation process targets a Spring Boot-based platform designed with modular components to promote a clear separation of concerns and facilitate future extensibility. As depicted in Figure 10, this platform adheres to a layered architectural style and integrates the Drools rules engine to dynamically manage business logic and decision-making processes.

- **Client:** Constitutes the client application, responsible for sending requests to the server and receiving corresponding responses.

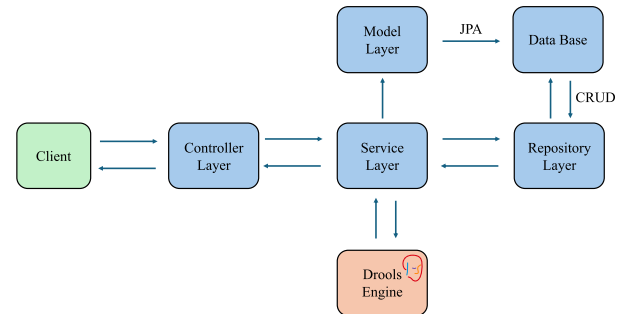


FIGURE 10. Target platform architecture.

- **Controller Layer:** Serves as the intermediary, processing incoming requests, coordinating interactions with the business logic, and returning appropriately formatted responses to the client.
- **Service Layer:** Contains the core business logic and integrates the Drools engine to handle complex business rules, ensuring accurate evaluation and execution.
- **Repository Layer:** Provides an abstraction for database interactions, offering streamlined CRUD operations and guaranteeing data consistency.
- **Model Layer:** Defines the structural representation of the domain entities, enabling organized data manipulation and use throughout the system.

V. CASE STUDY: DSS FOR SMART TOURISM

A. CASE STUDY DESCRIPTION

The purpose of this case study is to design and implement a DSS capable of managing three categories of tourism-related entities: hotels, natural attractions, and cafés. This DSS is intended to analyze the attributes of these entities, apply predefined business rules, and generate actionable insights to inform decision-making processes. In addition, the system

TABLE 1. Case study description.

Rule Name	Action Description	Condition	Attribute	Operator
HIGH_POPULARITY	Identify entities with high popularity.	The value of the attribute popularityRating must be greater than 0.9.	popularityRating	Greater_Than
LOW_CAPACITY	Flag entities with low capacity.	The value of the attribute capacity must be less than 100.	capacity	Less_Than
AFFORDABLE_ATTRACTION	Identify affordable attractions.	The value of the attribute price must be less than or equal to 10.	price	Less_Than_or_equals

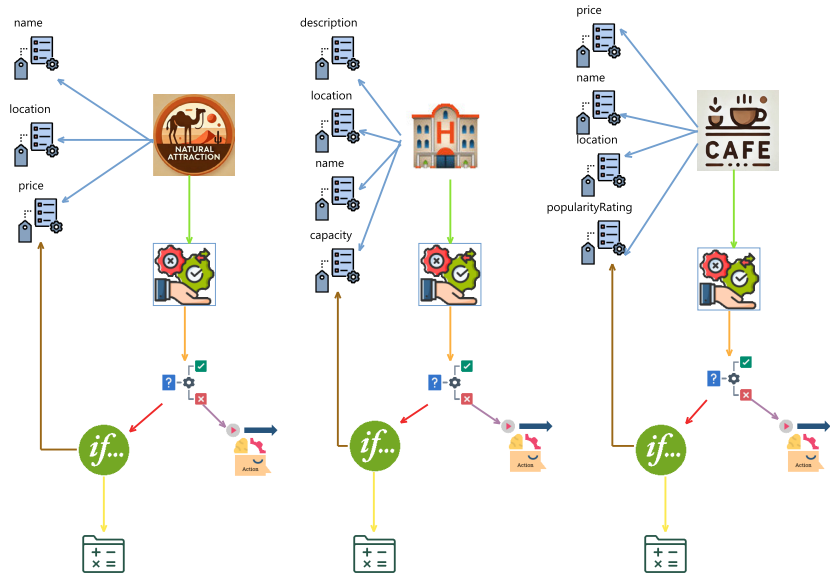


FIGURE 11. Conceptual DSS model.

provides CRUD (Create, Read, Update, Delete) functionalities to facilitate efficient data handling. The underlying logic is governed by a set of business rules, summarized in Table 1, that systematically evaluate the critical attributes of each entity type against defined conditions. These rules serve to guide various decisions, ensuring the system’s responsiveness to the operational needs of tourism managers.

B. MODELING PHASE OF THE DSS USING THE DSL

Within our case study, the DSS modeling phase relies on a dedicated DSL to simplify how system components and business rules are defined and documented. The resulting model, depicted in Figure 11 and commonly termed the PIM, is constructed using the DSL and acts as the cornerstone for subsequent code generation. This model consolidates all relevant entities, attributes, and rules, ensuring that the DSS precisely adheres to the case study’s established requirements.

C. CODE GENERATION

Following the execution of the transformation engine, the resulting code is displayed in Figure 12. This generated code adheres to JEE architectural principles and incorporates the fundamental layers; model, controller, service, and

interface; to maintain clear separation of concerns. Figure 12 presents the structure of the generated code and shows the organization of the different layers and their respective packages. To illustrate the details of the transformation, figure 13 highlights an example of a model layer class. In addition, Figure 14 gives an overview of a generated DRL file, which encapsulates the rule-based decision-making logic derived from the transformation process.

D. CONFIGURATION AND TESTING

This phase ensures that the generated DSS is correctly configured, and functions as intended. This phase involves minimal human intervention, in particular for configuring the necessary dependencies and carrying out tests. Figure 15 illustrates the Swagger interface for the DSS, showcasing the generated controllers such as HotelController, NaturalAttractionController, and CafeController. Each controller exposes CRUD operations and rule-based endpoints, aligning with the system requirements.

VI. VALIDATION OF THE APPROACH

A. VALIDATION PROTOCOL

To assess the impact of our approach on development efficiency, we conducted a comparative study focused on

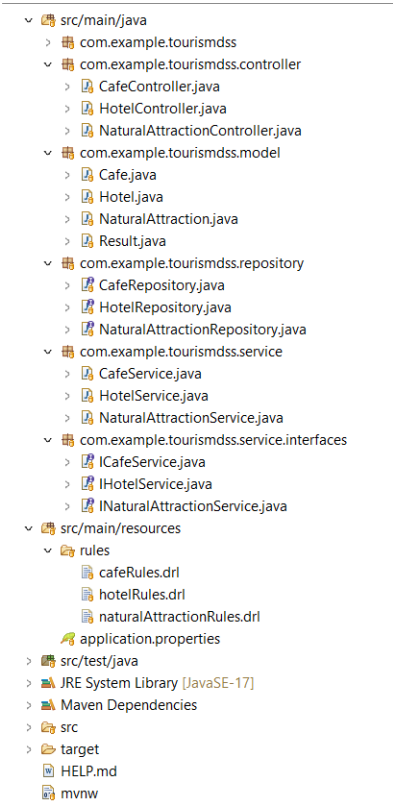


FIGURE 12. The structure of the code generated for the DSS.

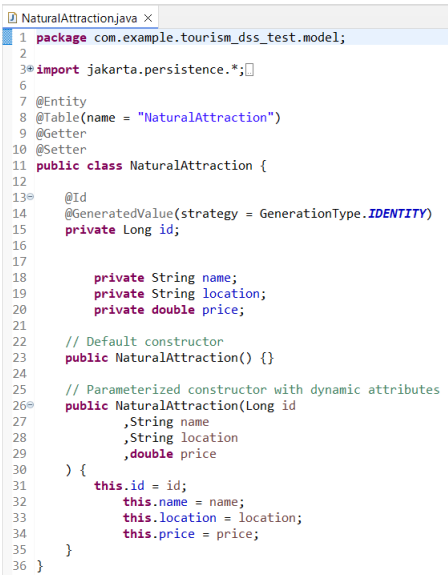


FIGURE 13. Example of generated java class.

measuring the reduction in development time. The validation aimed to assess the reduction in development time achieved by utilizing the proposed DSL and code generator for DSS in the tourism sector, compared to traditional development method. The time required for training and familiarization with the technologies or modeling environment was not

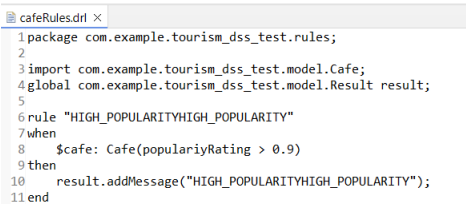


FIGURE 14. Example of a generated rules file.

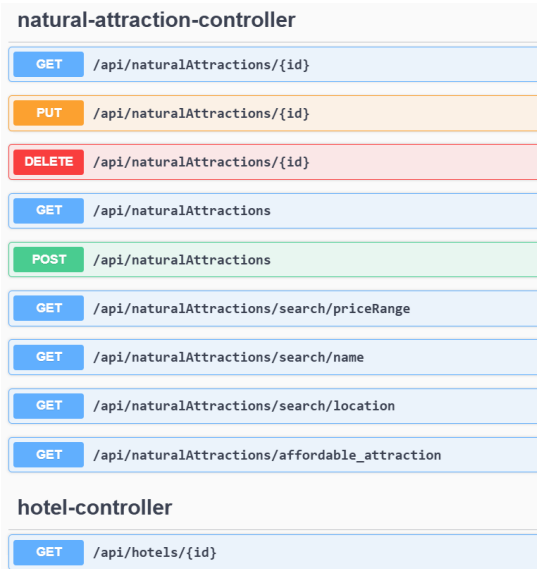


FIGURE 15. Swagger interface showing the generated controllers and their corresponding endpoints.

included in the measured time for either approach. The validation involved two parallel development scenarios:

- 1) Traditional development: In this scenario, the DSS was implemented manually using Java, Spring Boot, and Drools. The developers performed all tasks manually, including domain entities and the implementation of rule logic. The following environment was used:
 - Operating System: Windows 11
 - IDE: Eclipse IDE for Java and DSL Developers – 2024-06
 - Tools and Frameworks: Spring Boot, Drools 8.38.0.
- 2) DSL-based development: In this scenario, the same DSS was developed using the proposed DSL approach. The system was modeled through a graphical editor, and the corresponding code (including Drools rules) was generated automatically using predefined Acceleo templates. Manual intervention was limited to configuration and test tasks. The environment was as follows:
 - Operating System: Windows 11
 - IDE: Eclipse IDE for Java and DSL Developers – 2024-06
 - Tools and Frameworks: Sirius 7.4.0, Acceleo 3.7.12, Spring Boot.

TABLE 2. Development time in traditional approach.

Participant	Development Time (Td)
Participant 1	2h 35min
Participant 2	2h 15min
Participant 3	2h 20min
Total	7h
Average	2h 32min

The same DSS specification was used in both cases, and the tasks were performed by graduate students familiar with software development. In the traditional approach, participants were responsible for implementation, testing, and debugging. In contrast, the DSL-based approach limited manual effort to high-level modeling and basic configuration tasks, with the remainder handled by the code generation engine.

B. RESULTS ANALYSIS

An examination of the experimental data reveals pronounced discrepancies between the traditional development process and the DSL-based approach, as outlined in Table 2 and Table 3. In the traditional method, the mean development time was 2 hours and 32 minutes per participant, accumulating 7 hours in total for the three participants. Notably, individual completion times varied from 2 hours and 15 minutes to 2 hours and 35 minutes, underscoring the challenges posed by repetitiveness, such as integrating components and diagnosing errors, factors that prolonged overall development. In stark contrast, Table 3 demonstrates a substantial decrease in development time when utilizing the DSL-based approach. By employing the proposed DSL, participants finished the modeling phase in an average of 10 minutes and 18 seconds, with the automated code generation phase requiring only an additional 5 minutes. This led to an overall average of 15 minutes and 17 seconds, a clear indication of the efficiency gains achieved through automation and domain-specific modeling. Table 4 offers a comparative perspective of both methods, spotlighting the significant benefits afforded by the DSL-based framework. Drawing on averages from Table 2 and Table 3, the DSL approach reduced total development time by over 89%. Specifically, the conventional method demanded 140 minutes on average, whereas the DSL approach required just 15 minutes and 18 seconds, a net saving of 124 minutes and 42 seconds, translating to an 89.28% improvement. A similar pattern emerges in the aggregate figures: the DSL approach totaled approximately 45 minutes and 53 seconds, far below the 420 minutes recorded for the traditional process, yielding a saving of 374 minutes and 7 seconds (a reduction of 89.04%). These results underscore the enhanced efficiency and reliability offered by the DSL-based approach. By leveraging automation and structured abstractions at the domain level, developers can expedite DSS construction, ensuring quicker implementation alongside greater consistency and maintainability.

TABLE 3. Development time in DSL-Based approach.

Participant	Modeling Time	Code Generation Time	Total Time
Participant 1	09min 09s	3min 55s	13min 04s
Participant 2	09min 34s	4min 08s	13min 42s
Participant 3	12min 10s	6min 57s	18min 07s
Total	30min 53s	15min	45min 53s
Average	10min 18s	5min	15min 18s

TABLE 4. Global comparison of development times.

Metric	Traditional Approach (min)	DSL-Based Approach (min)	Difference (min)	Savings (%)
Total Development Time	420min	45min 53s	374min 07s	89.04%
Average Development Time	140min	15min 18s	124min 42s	89.28%

VII. LIMITATIONS AND CHALLENGES

While the proposed DSL-based framework demonstrates promising results in terms of development effort reduction and code automation, several limitations and open challenges remain.

The current DSL implementation focuses on simple, standalone rules and does not yet support advanced Drools features such as rule salience, agenda groups, logical insertions, or accumulations. This design choice was made to ensure ease of use for non-technical users and to validate the core code generation pipeline. Consequently, the evaluation concentrated on development effort reduction, and runtime performance metrics, such as execution time, latency, or rule evaluation complexity, were not included. As the DSL evolves to support more complex rule logic, future work will consider both the integration of advanced Drools features and the incorporation of runtime benchmarking to assess system performance at scale.

In terms of robustness, the current framework includes OCL constraints to ensure the structural validity of business rules, such as verifying that each rule contains a defined condition and an associated action. However, semantic rule conflicts (e.g., overlapping or contradictory logic) are not automatically detected at this stage. While Drools offers internal mechanisms like salience and agenda groups to manage execution priorities, comprehensive conflict detection and error-handling strategies at the modeling level remain areas for future enhancement.

The current implementation is tightly coupled with the Drools rule engine, which is Java-based. This choice was made to simplify decision rule management and to take advantage of Drools' mature integration with model-driven engineering tools. However, this introduces platform dependency that may limit adoption by teams working outside the Java ecosystem. In future work, we aim to extend

the framework to support alternative rule engines, enabling broader interoperability and deployment flexibility.

VIII. CONCLUSION

This work presented a model-driven approach for developing DSS in the smart tourism domain, centered on a DSL and supported by an automated code generation process. Our main contributions include the design of a domain-specific metamodel for DSS, the implementation of a graphical DSL for modeling decision logic and tourism-related entities, and the integration of a code generator that produces executable applications based on Spring and Drools.

Through a comparative case study, we demonstrated that our approach significantly reduces development time compared to traditional manual coding methods. By abstracting technical complexity and automating repetitive implementation tasks, the DSL enables developers to focus on decision logic rather than low-level system configuration. The results confirm the effectiveness and efficiency of DSL-based development for rule-oriented systems.

Currently, the DSL supports only simple and independent rules, without advanced Drools features. This simplification was intentional to prioritize usability and validate the automation pipeline. However, supporting complex rule dependencies is essential for scaling the system to real-world, multi-layered decision scenarios. Addressing this limitation is part of our future development roadmap, which also includes the integration of machine learning-based decision models and natural language interfaces to enhance system intelligence, accessibility, and adaptability. In addition, we plan to extend the DSL framework to other domains such as finance, logistics, and public administration.

REFERENCES

- [1] M. Kulkarni, A. Wadhaval, and P. Shinde, "Decision support system," *Int. J. Eng. Trends Technol.*, vol. 4, no. 4, pp. 671–675, Apr. 2013.
- [2] D. Das, B. Kim, and S. Chernova, "Subgoal-based explanations for unreliable intelligent decision support systems," in *Proc. 28th Int. Conf. Intell. User Interfaces*, Mar. 2023, pp. 240–250.
- [3] Microsoft Azure. (2025). *Azure Machine Learning*. Accessed: Feb. 18, 2025. [Online]. Available: <https://azure.microsoft.com/en-us/products/machine-learning>
- [4] IBM. (2025). *IBM Decision Optimization*. Accessed: Feb. 18, 2025. [Online]. Available: <https://www.ibm.com/decision-optimization>
- [5] Drools. (2025). *Drools—Business Rules Management System*. Accessed: Feb. 18, 2025. [Online]. Available: <https://www.drools.org/>
- [6] Tableau. (2025). *Tableau—Business Intelligence and Analytics*. Accessed: Feb. 12, 2025. [Online]. Available: <https://www.tableau.com/>
- [7] Microsoft Power BI. (2025). *Microsoft Power BI—Business Analytics Tools*. Accessed: Feb. 18, 2025. [Online]. Available: <https://www.microsoft.com/fr-fr/power-platform/products/power-bi>
- [8] J. Dorcic, J. Komsic, and S. Markovic, "Mobile technologies and applications towards smart tourism—state of the art," *Tourism Rev.*, vol. 74, no. 1, pp. 82–103, Feb. 2019.
- [9] N. Wise and H. Heidari, "Developing smart tourism destinations with the Internet of Things," in *Big Data and Innovation in Tourism, Travel, and Hospitality*. London, U.K.: Springer, 2019, pp. 21–29.
- [10] L. Benaddi, C. Ouaddi, A. Jakimi, and B. Ouchao, "A systematic review of chatbots: Classification, development, and their impact on tourism," *IEEE Access*, vol. 12, pp. 78799–78810, 2024.
- [11] A. Souha, L. Benaddi, C. Ouaddi, E. M. Bouziane, and A. Jakimi, "A new decision support system for enhancing tourism destination management and competitiveness," in *Proc. 11th Int. Conf. Wireless Netw. Mobile Commun. (WINCOM)*, Jul. 2024, pp. 1–6.
- [12] D. Buhalis and A. Amaranggana, "Smart tourism destinations," in *Proc. Int. Conf. Dublin Inf. Commun. Technol. Tourism*, Dublin, Ireland. Cham, Switzerland: Springer, Jan. 2013, pp. 553–564.
- [13] A. A. Abdul-Kareem, Z. T. Fayed, S. Rady, S. Amin, and B. M. Nema, "Advances in decision support systems' design aspects: Architecture, applications, and methods," *Int. J. Intell. Comput. Inf. Sci.*, vol. 23, no. 2, pp. 74–104, Jun. 2023.
- [14] A. Souha, L. Benaddi, C. Ouaddi, E. M. Bouziane, and A. Jakimi, "Towards a decision support system for tourism sector in daraa-tafilalet," in *Proc. 4th Int. Conf. Innov. Res. Appl. Sci., Eng. Technol. (IRASET)*, May 2024, pp. 1–5.
- [15] J. Li, J. Dai, A. Issakhov, S. F. Almojil, and A. Souri, "Towards decision support systems for energy management in the smart industry and Internet of Things," *Comput. Ind. Eng.*, vol. 161, Nov. 2021, Art. no. 107671.
- [16] G. van Valkenhoef, T. Tervonen, T. Zwinkels, B. de Brock, and H. L. Hillege, "ADDIS: A decision support system for evidence-based medicine," *Decis. Support Syst.*, vol. 55, no. 2, pp. 459–475, May 2013.
- [17] A. Sopandi, "Optimization of decision support system in investment risk management with firefly algorithm," *J. Appl. Data Sci.*, vol. 4, no. 2, pp. 93–102, May 2023.
- [18] G. Fenu and F. M. Mallocci, "DSS LANDS: A decision support system for agriculture in Sardinia," *HighTech Innov. J.*, vol. 1, no. 3, pp. 129–135, Sep. 2020.
- [19] S. Jana, R. Majumder, P. P. Menon, and D. Ghose, "Decision support system (DSS) for hierarchical allocation of resources and tasks for disaster management," in *Proc. Operations Res. Forum*, vol. 3, Jul. 2022, p. 37.
- [20] L. Benaddi, C. Ouaddi, L. Naimi, A. Souha, A. Jakimi, and B. Ouchao, "A decision support system for enhancing transportation services using aspect-based sentiment analysis," in *Proc. Int. Conf. Decis. Aid Sci. Appl. (DASA)*, Dec. 2024, pp. 1–5.
- [21] A. Souha, L. Benaddi, C. Ouaddi, E. M. Bouziane, and A. Jakimi, "Towards a framework for building a decision support system for customer satisfaction assessment in tourism sector," in *Proc. Medit. Smart Cities Conf. (MSCC)*, May 2024, pp. 1–5.
- [22] R. Liu and J. Li, "Application of decision support system in tourism development of Qingjiang river," in *Proc. Int. Conf. New. Finance Develop. Wuhan, China: Scientific Research Publishing*, 2010, pp. 271–276. [Online]. Available: <https://www.scirp.org/pdf/13-158.pdf>
- [23] A. Savvopoulos, G. Kalogeras, A. Mariotti, C. Bernini, C. Alexakos, and A. Kalogeras, "Data analytics supporting decision making in the tourism sector," in *Proc. 1st Int. Conf. Societal Autom. (SA)*, Sep. 2019, pp. 1–6.
- [24] C. W. Holsapple, "DSS architecture and types," in *Handbook on Decision Support Systems 1*, 2008, pp. 163–189.
- [25] J. E. Aronson, T.-P. Liang, and R. V. MacCarthy, *Decision Support Systems and Intelligent Systems*, vol. 4. Upper Saddle River, NJ, USA: Pearson, 2005.
- [26] R. Baggio and L. Caporarello, "Decision support systems in a tourism destination: Literature survey and model building," in *U. Proc. 2nd Conf. Italian Chapter AIS*, Verona, Italy, Jan. 2005, pp. 1–15.
- [27] D. Jung, V. Tran Tuan, D. Quoc Tran, M. Park, and S. Park, "Conceptual framework of an intelligent decision support system for smart city disaster management," *Appl. Sci.*, vol. 10, no. 2, p. 666, Jan. 2020.
- [28] M. Soori, F. K. G. Jough, R. Dastres, and B. Arezoo, "AI-based decision support systems in industry 4.0, a review," *J. Economy Technol.*, vol. 80, pp. 1–15, Aug. 2024.
- [29] H. J. Ko, "A DSS approach with fuzzy AHP to facilitate international multimodal transportation network," *KMI Int. J. Maritime Affairs Fisheries*, vol. 1, no. 1, pp. 51–70, Jun. 2009.
- [30] Y. S. Bagi, S. Suyono, and M. F. Tomatala, "Decision support system for high achieving students selection using AHP and TOPSIS," in *Proc. 2nd Int. Conf. Cybern. Intell. Syst. (ICORIS)*, Oct. 2020, pp. 1–5.
- [31] O. Havrylenko, K. Dergachov, V. Pavlikov, S. Zhyla, O. Shmatko, N. Ruzhentsev, A. Popov, V. Volosyuk, E. Tserne, and M. Zaliskiyi, *Decision Support System Based on the ELECTRE Method*. Cham, Switzerland: Springer, 2022, pp. 295–304.
- [32] G. Kostopoulos, G. Davrazos, and S. Kotsiantis, "Explainable artificial intelligence-based decision support systems: A recent review," *Electronics*, vol. 13, no. 14, p. 2842, Jul. 2024.

- [33] N. Kamboj, K. Metcalfe, C. H. Chu, and A. Conway, "Designing the user interface of a nitroglycerin dose titration decision support system: User-centered design study," *Appl. Clin. Informat.*, vol. 15, no. 3, pp. 583–599, May 2024.
- [34] M. Özbayrak and R. Bell, "A knowledge-based decision support system for the management of parts and tools in fms," *Decis. Support Syst.*, vol. 35, no. 4, pp. 487–515, Jul. 2003.
- [35] S. Elnagar and H. R. Weistroffer, "Introducing knowledge graphs to decision support systems design," in *Proc. EuroSymp. Syst. Anal. Des.*, Gdansk, Poland, Jan. 2019, pp. 3–11.
- [36] M. Mardison, A. Ramadhanu, L. N. Rani, and S. Enggari, "Hybrid. DSS for recommendations of halal culinary tourism west Sumatra," *IAES Int. J. Artif. Intell. (IJ-AI)*, vol. 10, no. 2, pp. 273–283, Jun. 2021.
- [37] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," 2019, *arXiv:1912.01703*.
- [38] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, Sep. 2020.
- [39] I. Bider and A. Jalali, "Agile business process development: Why, how and when—Applying Nonaka's theory of knowledge transformation to business process development," *Inf. Syst. E-Bus. Manage.*, vol. 14, no. 4, pp. 693–731, Nov. 2016.
- [40] M. Dumas, M. La Rosa, J. Mendling, and H. A. Reijers, *Fundamentals of Business Process Management*, vol. 1. Cham, Switzerland: Springer, 2013.
- [41] Qlik. (2025). *QlikView—Business Intelligence and Data Visualization*. Accessed: Feb. 18, 2025. [Online]. Available: <https://www.qlik.com/us/products/qlikview>
- [42] MindMajix. (2025). *What is Drools?*. Accessed: Feb. 18, 2025. [Online]. Available: <https://mindmajix.com/what-is-drools>
- [43] J. Bézivin and I. Kurtev, "MDE: Foundations, standards and tools," *ECDMA-FA*, vol. 4066, pp. 346–360, Nov. 2005.
- [44] J. Bézivin, *Model Driven Engineering: An Emerging Technical Space* (Lecture Notes in Computer Science), vol. 4143. Cham, Switzerland: Springer, 2006, pp. 36–64, doi: [10.1007/11877028](https://doi.org/10.1007/11877028).
- [45] C. Ouaddi, L. Benaddi, E. M. Bouziane, A. Jakimi, A. Chehri, and R. Saadane, "DSL-Driven approaches and metamodels for chatbot development: A systematic literature review," *Expert Syst.*, vol. 42, no. 2, p. 13787, Feb. 2025.
- [46] C. Ouaddi, L. Benaddi, A. Souha, L. Naimi, E. M. Bouziane, and A. Jakimi, "A metamodel for a DSL to design decision support systems in smart tourism," in *Proc. Int. Conf. Decis. Aid Sci. Appl. (DASA)*, Dec. 2024, pp. 1–5.
- [47] R. France and B. Rumpe, "Model-driven development of complex software: A research roadmap," in *Proc. Future Softw. Eng. (FOSE)*, May 2007, pp. 37–54.
- [48] H. Gannud, H. Wu, and J. Timoney, "Applying a MDE approach to a healthcare environment: A case study of an AE dept," in *Proc. 28th Irish Signals Syst. Conf. (ISSC)*, Jun. 2017, pp. 1–7.
- [49] L. Benaddi, C. Ouaddi, A. Souha, A. Jakimi, and B. Ouchao, "Model-driven engineering to develop chatbots for smart tourism," in *Proc. Int. Conf. Global Aeronaut. Eng. Satell. Technol. (GAST)*, Apr. 2024, pp. 1–5.
- [50] B. Ouni, C. Aussagues, S. Dhoubi, and C. Mraïdha, "Model-driven architectural framework towards safe and secure nuclear power reactors," *Sensors*, vol. 21, no. 15, p. 5136, Jul. 2021.
- [51] S. Mirachi, V. da Costa Guerra, A. M. da Cunha, L. A. V. Dias, and E. Villani, "Applying agile methods to aircraft embedded software: An experimental analysis," *Softw., Pract. Exper.*, vol. 47, no. 11, pp. 1465–1484, Nov. 2017.
- [52] P. Mohagheghi, M. A. Fernandez, J. A. Martell, M. Fritzsche, and W. Gilani, *MDE Adoption in Industry: Challenges and Success Criteria*. Cham, Switzerland: Springer, 2009, pp. 54–59.
- [53] J. Miller and J. Mukerji, "MDA guide version 1.0. 1," Object Management Group, Needham, MA, USA, Tech. Rep. omg/2003-06-01, 2003.
- [54] A. Z. Javed, P. A. Strooper, and G. N. Watson, "Automated generation of test cases using model-driven architecture," in *Proc. 2nd Int. Workshop Autom. Softw. Test*, May 2007, pp. 1–16.
- [55] S. Cook, "Domain-specific modeling and model-driven architecture," *MDA J., Model Driven Archit. Straight Masters*, vol. 1, pp. 3–10, Dec. 2004.
- [56] X. Blanc and O. Salvatori, *MDA En Action: Ingénierie Logicielle Guidée Par Les Modèles*. Paris, France: Éditions Eyrolles, 2011.
- [57] *MOF QVT Final Adopted Specification*, document ptc/05-11-01, Object Management Group, 2005. [Online]. Available: <http://www.omg.org/docs/ptc/05-11-01.pdf>
- [58] F. Jouault and B.-K. Loukil, "Model-to-model transformation with ATL," Tech. Rep., 2008.
- [59] S. Beydeda and V. Gruhn, *Model-Driven Software Development*. New York, NY, USA: Springer, 2005.
- [60] C. Ouaddi, L. Benaddi, A. Souha, and A. Jakimi, "A comparative and analysis study for recommending a chatbot development tool," in *Proc. Int. Conf. Global Aeronaut. Eng. Satell. Technol. (GAST)*, Apr. 2024, pp. 1–6.
- [61] S. Pérez-Soler, E. Guerra, and J. de Lara, "Creating and migrating chatbots with conga," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng., Companion*, May 2021, pp. 37–40.
- [62] C. Ouaddi, L. Benaddi, E. M. Bouziane, A. Jakimi, A. Chehri, and R. Saadane, "A sketch of DSL and code generator for accelerating chatbot development," *Proc. Comput. Sci.*, vol. 246, pp. 3585–3594, Nov. 2024.
- [63] A. Souha, C. Ouaddi, L. Benaddi, L. Naimi, E. Mahi Bouziane, and A. Jakimi, "MDE-based approach for accelerating the development of recommender systems in smart tourism," *IEEE Access*, vol. 13, pp. 31615–31629, 2025.
- [64] B. Ulitin and E. Babkin, "Ontology-based reconfigurable DSL for planning technical services," *IFAC-PapersOnLine*, vol. 52, no. 13, pp. 1138–1144, 2019.
- [65] M. Saxena, S. Song, and L. Sha, "Medik: Towards safe guideline-based clinical decision support," in *Proc. Formal Methods Computer-Aided Design (FMCAD)*, Oct. 2023, pp. 306–317.
- [66] A. Khakpour, A. Vázquez-Ingelmo, R. Colomo-Palacios, F. J. García-Peñalvo, and A. Martini, "ModelViz: A model-driven engineering approach for visual analytics system design," *IEEE Access*, vol. 12, pp. 42667–42682, 2024.
- [67] J.-N. Mazón and J. Trujillo, "An MDA approach for the development of data warehouses," *Decis. Support Syst.*, vol. 45, no. 1, pp. 41–58, Apr. 2008.
- [68] M. Erwig and P. Kumar, "MADMAX: A DSL for explanatory decision making," in *Proc. 20th ACM SIGPLAN Int. Conf. Generative Program., Concepts Exper.*, Oct. 2021, pp. 144–155.
- [69] L. Naimi, C. Ouaddi, L. Benaddi, A. Souha, and A. Jakimi, "Towards a domain model of smart tourism based on the 6As framework," in *Proc. Int. Conf. Smart City Appl.*, Jan. 2025, pp. 278–287.
- [70] D. Buhalis, "Marketing the competitive destination of the future," *Tourism Manage.*, vol. 21, no. 1, pp. 97–116, Feb. 2000.
- [71] M. C. Er, "Decision support systems: A summary, problems, and future trends," *Decis. Support Syst.*, vol. 4, no. 3, pp. 355–363, Sep. 1988.
- [72] G. M. Marakas, *Decision Support Systems in the 21st Century*, vol. 134. Upper Saddle River, NJ, USA: Prentice-Hall, 2003.
- [73] K. C. Kang, S. Cohen, J. A. Hess, W. E. Novak, and A. S. Peterson, "Feature-oriented domain analysis (FODA) feasibility study," Carnegie Mellon University, Pittsburgh, PA, USA, Tech. CMU/SEI-90-TR-021, 1990. [Online]. Available: <https://apps.dtic.mil/sti/citations/ADA235785>



ADNANE SOUHA (Member, IEEE) received the master's degree in software engineering and information systems from the Faculty of Science and Techniques of Errachidia, Meknes, Morocco, in 2021. Currently, he is a Ph.D. Student in the GL-ISI team with the Faculty of Sciences and Techniques of Errachidia, University of Moulay Ismail. He is also a Teacher with the Primary School, Ministry of Education, Morocco. His research interests include recommender systems, software modeling, DSLs, natural language processing, and deep learning.



driven engineering (MDE), model-driven architecture (MDA), domain-specific languages (DSLs), chatbot development, modeling, deep learning, smart systems, and tourism applications.

CHARAF OUADDI (Member, IEEE) received the Ph.D. degree in computer science, specializing in Software Engineering and Artificial Intelligence, from the Faculty of Sciences and Techniques, Moulay Ismail University, Errachidia, Morocco, in 2024. He received the master's degree in software engineering and information systems from Moulay Ismail University, in 2020. He is also a computer science teacher at the Ministry of Education in Morocco. His research interests include model-



interfaces, artificial intelligence, and smart applications.

EL MAHI BOUZIANE received the Ph.D. degree in software engineering from the Faculty of Science and Techniques of Errachidia, Morocco, in 2021. He is currently a Professor of computer sciences and engineering with the Department of Computer Sciences, Faculty of Sciences and Technology of Errachidia, University of Moulay Ismail. His research interests include model-driven engineering (UML and MDA), transformational approaches, design and development of user



languages (DSLs) for developing chatbots, software engineering, artificial intelligence, intelligent systems, and tourism applications.

LAMY BENADDI (Member, IEEE) received the master's degree in software engineering and information systems and the Ph.D. degree in software engineering and artificial intelligence from the Faculty of Sciences and Techniques of Errachidia, University of Moulay Ismail, Meknes, Morocco, in 2020 and 2025, respectively. She is currently a Computer Science Teacher with the College Level for the Ministry of Education, Morocco. Her research interests include domain-specific



interests include software modernization, software modeling, model-driven engineering, and reverse engineering.

HAMZA ABDELMALEK received the Ph.D. degree in software engineering and artificial intelligence from the Faculty of Sciences and Techniques of Errachidia, University of Moulay Ismail, Meknes, Morocco, in 2024. Currently, he is a Software Engineer with the Ministry of Justice, Morocco, where he contributes to the digital transformation of the country's legal sector. He is also exploring the application of artificial intelligence in software modeling. His research



in the areas of software engineering and artificial intelligence, since 2010. His research interests include software engineering, artificial intelligence, data visualization, blockchain, the IoT, and smart applications. He is also a member of several scientific association. He has served as a member and the chairperson of steering committees and program committees for several international conferences, workshops, and symposiums.

ABDESLAM JAKIMI (Member, IEEE) is currently a Professor of computer sciences and engineering with the Department of Computer Sciences, Faculty of Sciences and Technology of Errachidia, University of Moulay Ismail. He is the Head of the Department of Computer Sciences, the President of the Scientific Association for Academic Researchers (ASCA), and the Chair of the International Conference in Computer Science and Applications (ICCSA). He has been teaching

...